

Programmazione 1

Corso c

05/11/2023

> Dagli array alle matrici

Alessandro Mazzei

alessandro.mazzei@unito.it

Dipartimento di Informatica

Università degli Studi di Torino



UNIVERSITÀ
DI TORINO

Outline

- Stringhe
- Ricerca Binaria in un array
- Aritmetica dei puntatori
- Array multidimensionali

Outline

- Stringhe
- Ricerca Binaria in un array
- Aritmetica dei puntatori
- Array multidimensionali

Stringhe in C

(Sec. 6.5)

- **char** string1[] = **"first"**;
- La stringa "first" contiene cinque caratteri più un **carattere nullo di terminazione di stringa**.
- Quindi, **string1** contiene in realtà sei elementi. La sequenza di escape che rappresenta il carattere nullo è **'\0'**.
- Tutte le stringhe terminano con questo carattere.
- Un array di caratteri che rappresenta una stringa deve essere sempre definito di dimensioni abbastanza grandi da contenere il numero di caratteri nella stringa e il carattere nullo che la termina.

Stringhe in C

```
char string1[] = "first";
```

- string1[0] è 'f'
- string1[3] è 's'
- string1[5] è '\0'

```
char string1[] = {'f', 'i', 'r', 's', 't', '\0'};
```

Capitolo 8 libro: "Caratteri e Stringhe"

- ... introduce le funzioni della Libreria Standard del C che facilitano l'elaborazione di caratteri, stringhe, righe di testo e blocchi di memoria. Il capitolo prende in esame le tecniche utilizzate per sviluppare editor, word processor, software per l'impaginazione e altri tipi di software per l'elaborazione del testo.
- È possibile implementare le manipolazioni di testi eseguite da funzioni per la formattazione dell'input/output come `printf` e `scanf` usando le funzioni esaminate in questo capitolo.

Stringhe in C

```
char string2[20];  
• scanf("%s", string2);  
• scanf("%19s", string2);
```

*È vostra responsabilità assicurarvi che l'array nel quale la stringa letta viene memorizzata possa contenere qualunque stringa che l'utente scrive alla tastiera. **La funzione scanf non controlla quanto è grande l'array; legge i caratteri finché non si incontra uno spazio, una tabulazione, un newline o un indicatore di end-of-file.** La stringa string2 non deve essere più lunga di 19 caratteri così da lasciare spazio al carattere nullo che la termina. Se l'utente scrive 20 o più caratteri, il programma può arrestarsi o creare un problema di vulnerabilità relativamente alla sicurezza noto come **overflow del buffer**. Per questa ragione, abbiamo usato la specifica di conversione %19s in modo che scanf legga un massimo di 19 caratteri e non scriva caratteri in memoria oltre la fine dell'array string2.*

Esercizio su Array/Stringa

```
1. // fig06_08.c
2. // Treating character arrays as
   strings.
3. #include <stdio.h>
4. #define SIZE 20
5.
6. // function main begins program
   execution
7. int main(void) {
8.     char string1[SIZE] = ""; //
   reserves 20 characters
9.     char string2[] = "string
   literal"; // reserves 15
   characters
10.
11.    // prompt for string from
   user then read it into array
   string1
12.    printf("%s", "Enter a string
   (no longer than 19 characters):
   ");
13.    scanf("%19s", string1); // input no
   more than 19 characters
14.
15.    // output strings
16.    printf("string1 is: %s\nstring2
   is: %s\n", string1, string2);
17.    puts("string1 with spaces between
   characters is:");
18.
19.    // output characters until null
   character is reached
20.    for (size_t i = 0; i < SIZE &&
   string1[i] != '\0'; ++i) {
21.        printf("%c ", string1[i]);
22.    }
23.
24.    puts("");
25. }
```

Outline

- Stringhe
- Ricerca Binaria in un array
- Aritmetica dei puntatori
- Array multidimensionali

6.10.1 Ricerca Lineare in un array

- Una ricerca lineare confronta ogni elemento dell'array con la chiave di ricerca.
- **L'array non è ordinato**, quindi è altrettanto probabile che il valore si trovi nel primo elemento o nell'ultimo.
- In media, il programma dovrà confrontare la chiave di ricerca con la metà degli elementi dell'array.
- Se il valore della chiave viene trovato, si restituisce l'indice dell'elemento nell'array, altrimenti -1

Ricerca Lineare in un array

```
int linearSearch(int a[], int lenA, int key) {  
    int ret = -1;  
    for (int i = 0; i < lenA && ret == -1; i++) {  
        if (a[i] == key) {  
            ret = i;  
        }  
    }  
    return ret;  
}
```

6.10.2 Ricerca Binaria in un array ordinato

- Per gli array di grandi dimensioni, la ricerca lineare è inefficiente.
- Se l'array è **ordinato**, possiamo usare la **ricerca binaria**, che elimina la metà degli elementi dell'array dopo ogni confronto.

6.10.2 Ricerca Binaria in un array ordinato

Algoritmo

- Individua l'elemento centrale dell'array e lo confronta con la chiave di ricerca:
 - Se sono uguali, viene trovata la chiave di ricerca e viene restituito il pedice dell'elemento. Se non sono uguali, cerca nella metà dell'array, ovvero:
 - Se la chiave di ricerca è inferiore all'elemento centrale, si cerca nella prima metà dell'array
 - altrimenti, si cerca nella seconda metà
- Ripetere su un quarto, poi su un ottavo, ..., dell'array originale fino arrivare a un elemento singolo

Prestazioni dell'algoritmo di ricerca binaria

- Nel caso peggiore, la ricerca di un array ordinato di 1024 elementi richiede solo 10 confronti con la ricerca binaria.
- Dividendo ripetutamente 1.024 per 2 si ottengono i valori 512, 256, 128, 64, 32, 16, 8 e 4, 2 e 1. Il numero 1.024 (2^{10}) viene diviso per 2 solo 10 volte per ottenere il valore 1.
- Un array ordinato di un miliardo di elementi ($\sim 2^{30}$) richiede al massimo **30** confronti per trovare la chiave di ricerca. Con la ricerca lineare **mediamente 500.000.000**

Ricerca Binaria in un array ordinato

```
1.  // fig06_15.c
2.  // Binary search of a sorted array.
3.  #include <stdio.h>
4.  #define SIZE 15
5.
6.  // Prototipi
7.  int binarySearch(const int b[], int key, size_t low, size_t high);
8.  void printHeader(void);
9.  void printRow(const int b[], size_t low, size_t mid, size_t high);
10.
11.
12. int main(void) {
13.     int a[SIZE] = {0}; // create array a
14.
15.     // create data
16.     for (size_t i = 0; i < SIZE; ++i) {
17.         a[i] = 2 * i;
18.     }
```

Ricerca Binaria in un array ordinato

```
19.  
20.     printf("%s", "Enter a number between 0 and 28: ");  
21.     int key = 0; // value to locate in array a  
22.     scanf("%d", &key);  
23.  
24.     printHeader();  
25.  
26.     // search for key in array a  
27.     int result = binarySearch(a, key, 0, SIZE - 1);  
28.  
29.     // display results  
30.     if (result != -1) {  
31.         printf("\n%d found at subscript %d\n", key, result);  
32.     }  
33.     else {  
34.         printf("\n%d not found\n", key);  
35.     }  
36. }  
37.
```

Ricerca Binaria in un array ordinato

```
38.  
39. int binarySearch(const int b[], int key, size_t low, size_t high) {  
40.     // loop until low subscript is greater than high subscript  
41.     while (low <= high) {  
42.         size_t middle = (low + high) / 2; //valore medio  
43.  
44.         // display subarray used in this loop iteration  
45.         printRow(b, low, middle, high);  
46.  
47.         // if key matches, return middle subscript  
48.         if (key == b[middle]) {  
49.             return middle;  
50.         }
```

Ricerca Binaria in un array ordinato

```
51.         else if (key < b[middle]) { //if key < b[middle]
52.             high = middle - 1; // elimino la parte alta
53.         }
54.         else { // key > b[middle], so adjust low
55.             low = middle + 1; // elimino la parte bassa
56.         }
57.     } // end while
58.
59.     return -1; // searchKey not found
60. }
```

Ricerca Binaria in un array ordinato

```
61.  
62. // Print a header for the output  
63. void printHeader(void) {  
64.     puts("\nSubscripts:");  
65.  
66.     // output column head  
67.     for (int i = 0; i < SIZE; ++i) {  
68.         printf("%3d ", i);  
69.     }  
70.  
71.     puts(""); // start new line of output  
72.  
73.     // output line of - characters  
74.     for (int i = 1; i <= 4 * SIZE; ++i) {  
75.         printf("%s", "-");  
76.     }  
77.  
78.     puts(""); // start new line of output  
79. }  
80.
```

Ricerca Binaria in un array ordinato

```
81. // Print one row of output showing the current
82. // part of the array being processed.
83. void printRow(const int b[], size_t low, size_t mid, size_t high) {
84.     // loop through entire array
85.     for (size_t i = 0; i < SIZE; ++i) {
86.         // display spaces if outside current subarray range
87.         if (i < low || i > high) {
88.             printf("%s", "    ");
89.         }
90.         else if (i == mid) { // display middle element
91.             printf("%3d*", b[i]); // mark middle value
92.         }
93.         else { // display other elements in subarray
94.             printf("%3d ", b[i]);
95.         }
96.     }
97.
98.     puts(""); // start new line of output
99. }
```

Ricerca Binaria in un array ordinato

Enter a number between 0 and 28: **25**

Subscripts:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

0	2	4	6	8	10	12	14*	16	18	20	22	24	26	28
								16	18	20	22*	24	26	28
												24	26*	28
												24*		

25 not found

Ricerca Binaria in un array ordinato

Enter a number between 0 and 28: **6**

Subscripts:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

0	2	4	6	8	10	12	14*	16	18	20	22	24	26	28
0	2	4	6*	8	10	12								

6 found at subscript 3

Outline

- Stringhe
- Ricerca Binaria in un array
- Aritmetica dei puntatori
- Array multidimensionali

Aritmetica dei puntatori

- Il linguaggio C maschera la complessità di accedere agli elementi di un array tramite la sintassi

`<nome array>[<indice>]`

- Cercheremo di comprendere come funziona il meccanismo referenziando le parole che compongono l'array senza parentesi «[]»
- Impareremo come utilizzare i puntatori per accedere a parole in aree di memoria arbitrarie

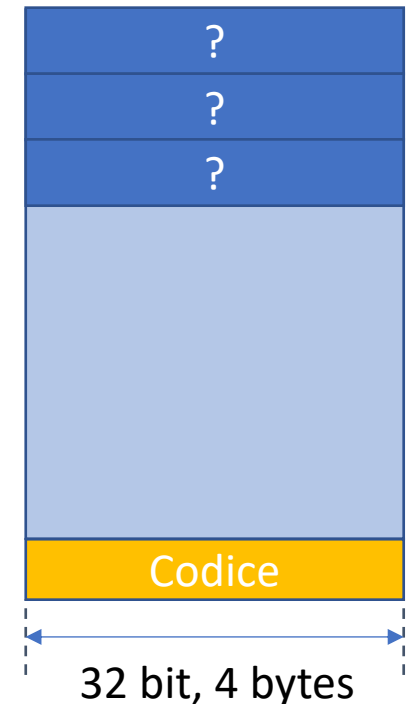
Aritmetica dei puntatori

- Le parentesi «[]» eseguono in modo trasparente due operazioni:
 1. calcolano l'indirizzo dell' i -esimo elemento aggiungendo all'indirizzo di base l'indice di cella moltiplicato per la dimensione del tipo base
 2. eseguono implicitamente la dereferenziazione di tale puntatore
- Utilizzeremo sempre le parentesi «[]» a meno che non ci sia una specifica motivazione per fare altrimenti
- Manipolando dati complessi, può essere necessario aggiornare punatatori in modo particolare

Ipotesi

- Ipotizzeremo di accedere ad un'area di memoria per la quale valgano le seguenti condizioni
 1. array di tipo omogeneo (tutti *int*, tutti *float*, etc.)
 2. parole che compongono l'array contigue in memoria (nessun «buco» fra elementi di un array)
 3. il primo elemento (elemento *base*) abbia indice 0
 4. tale elemento sia all'indirizzo più basso (cima allo stack)
- Quando un array è allocato in memoria, tutte le condizioni sopra sono automaticamente garantite

int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4



Accesso al primo elemento dell'array

- Il nome dell'array corrisponde all'indirizzo dell'elemento base

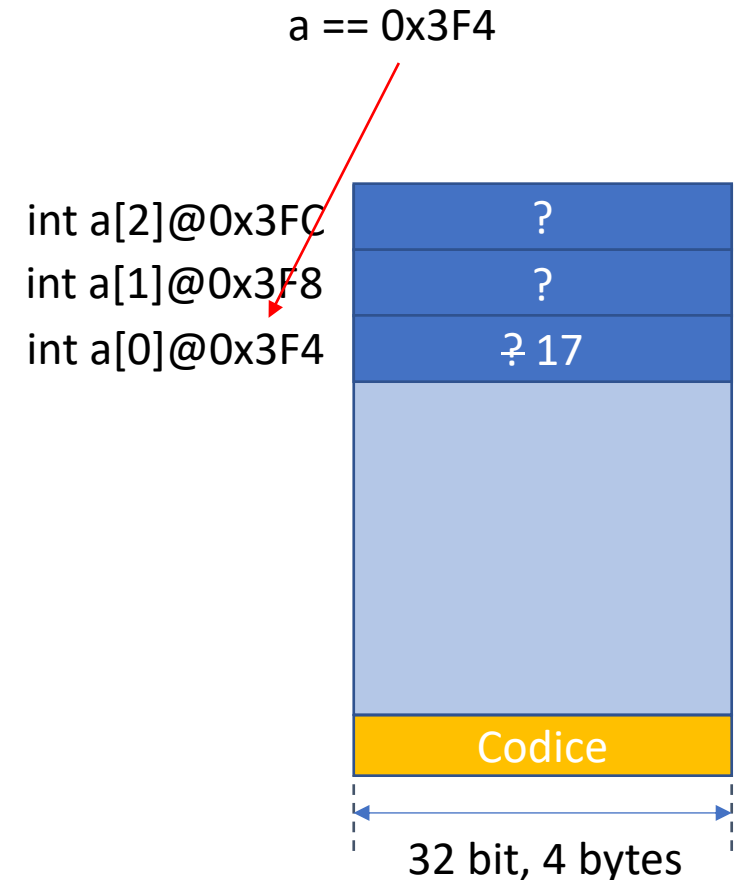
```
a == &a[0];
```

- Possiamo accedere al valore di `a[0]` per dereferenziazione in scrittura

```
*a = 17;
```

ed in lettura

```
printf("a[0] vale %d", *a);
```



`a[0] vale 17`

Accesso al primo elemento dell'array

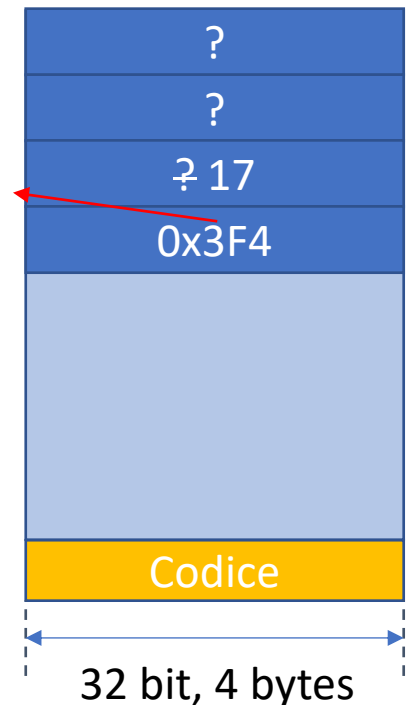
- Non lavoreremo mai direttamente con `a` bensì con una sua copia che modificheremo più avanti

```
int* pA = a;
```

- Ed esguiremo tutte le operazioni come visto prima

```
*pA = 17;  
printf("a[0] vale %d", *pA);
```

```
int a[2]@0x3FC  
int a[1]@0x3F8  
int a[0]@0x3F4  
int* pA@0x3F0
```



a[0] vale 17

Accesso al primo elemento dell'array

- Quindi, una volta dato

```
int* pA = a;
```

- I due assegnamenti seguenti sono equivalenti

```
a[0] = 17;
```

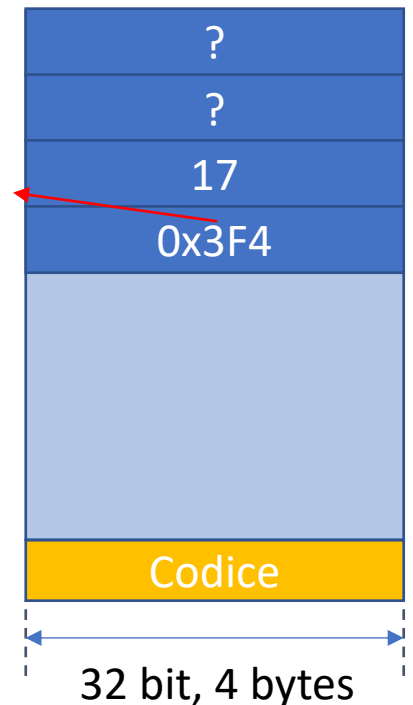
```
*pA = 17;
```

- Le due letture seguenti sono equivalenti

```
printf ("a[0] vale %d", a[0]);
```

```
printf ("a[0] vale %d", *pA);
```

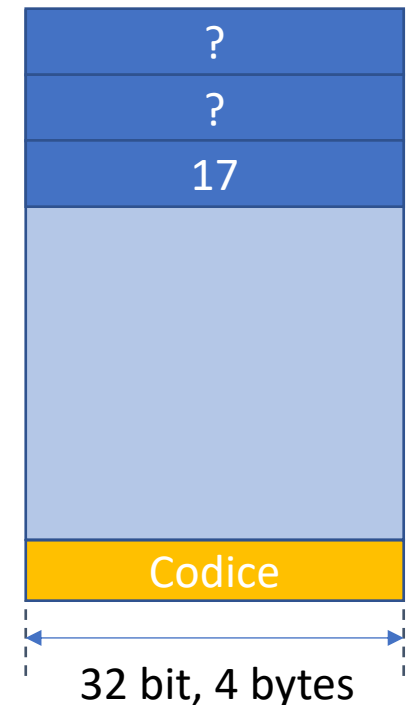
```
int a[2]@0x3FC  
int a[1]@0x3F8  
int a[0]@0x3F4  
int* pA@0x3F0
```



Accesso agli altri elementi dell'array

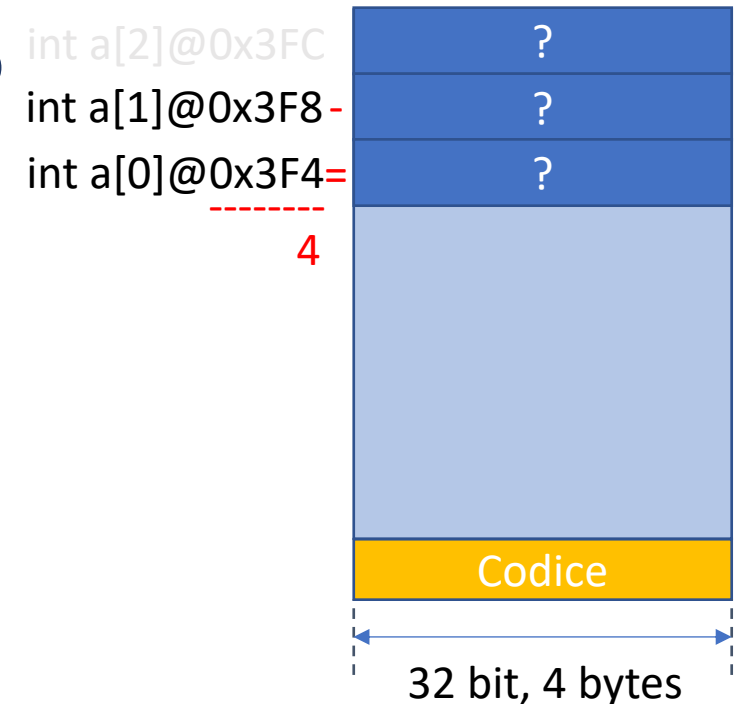
- Per accedere la valore degli altri elementi di *a* occorre prima calcolarne l'indirizzo per poterlo in seguito dereferenziare

int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4



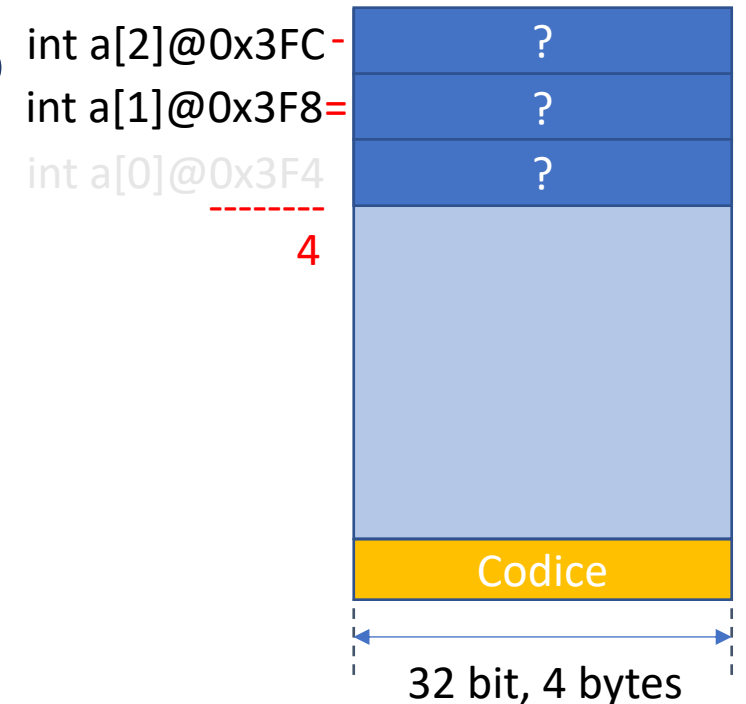
Accesso agli altri elementi dell'array

- Per accedere al valore degli altri elementi di a occorre prima calcolarne l'indirizzo per poterlo in seguito dereferenziare
- L'indirizzo di $a[1]$ è pari all'indirizzo di $a[0] + 4B$



Accesso agli altri elementi dell'array

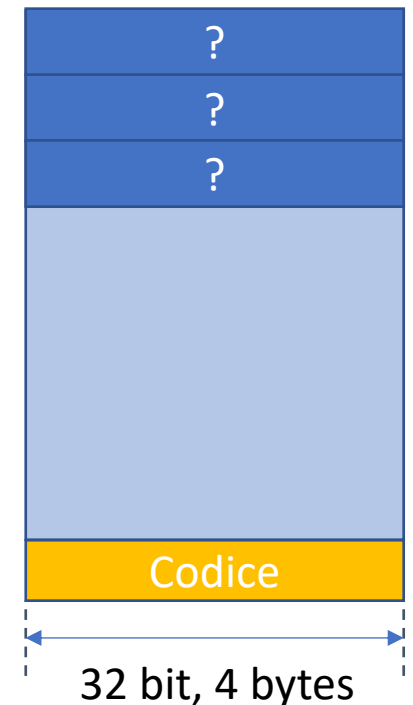
- Per accedere la valore degli altri elementi di a occorre prima calcolarne l'indirizzo per poterlo in seguito dereferenziare
- L'indirizzo di $a[1]$ è pari all'indirizzo di $a[0]+4B$
- L'indirizzo di $a[2]$ è pari all'indirizzo di $a[1]+4B$



Accesso agli altri elementi dell'array

- Per accedere la valore degli altri elementi di a occorre prima calcolarne l'indirizzo per poterlo in seguito dereferenziare
- L'indirizzo di $a[1]$ è pari all'indirizzo di $a[0] + 4B$
- L'indirizzo di $a[2]$ è pari all'indirizzo di $a[1] + 4B$
- L'indirizzo di $a[i+1]$ può essere calcolato come l'indirizzo di $a[i]$ incrementato di $4B$
- Dall'indirizzo di $a[0]$ ricavo l'indirizzo di $a[1]$, dal quale ricavo l'indirizzo di $a[2]$, etc.

int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4

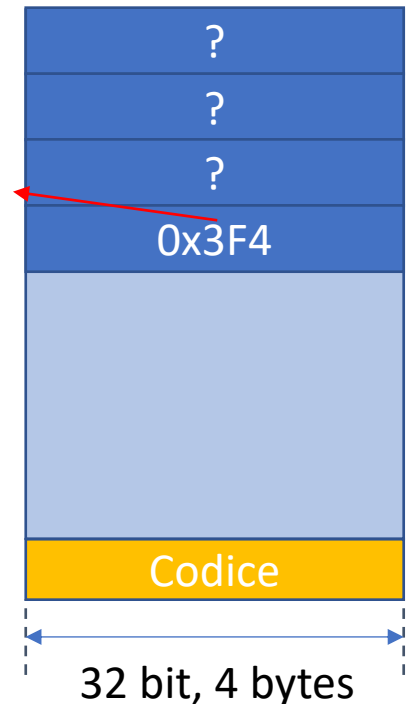


Accesso agli altri elementi dell'array

- Gli indirizzi sono tipi numerici positivi interi sui quali sono definiti gli operatori + e -
- Ipoteizzando che l'indirizzo di a[0] sia copiato in pA

`int* pA = a;`

`int a[2]@0x3FC`
`int a[1]@0x3F8`
`int a[0]@0x3F4`
`int* pA@0x3F0`



Accesso agli altri elementi dell'array

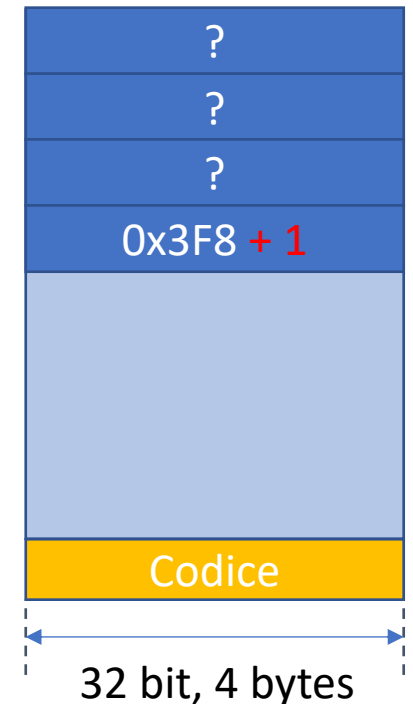
- Gli indirizzi sono tipi numerici positivi interi sui quali sono definiti gli operatori + e -
Ipotezzando che l'indirizzo di a[0] sia copiato in pA

`int* pA = a;`

l'indirizzo di a[1] in linguaggio C si calcola come

`pA = pA + 1;`

`int a[2]@0x3FC`
`int a[1]@0x3F8`
`int a[0]@0x3F4`
`int* pA@0x3F0`



Accesso agli altri elementi dell'array

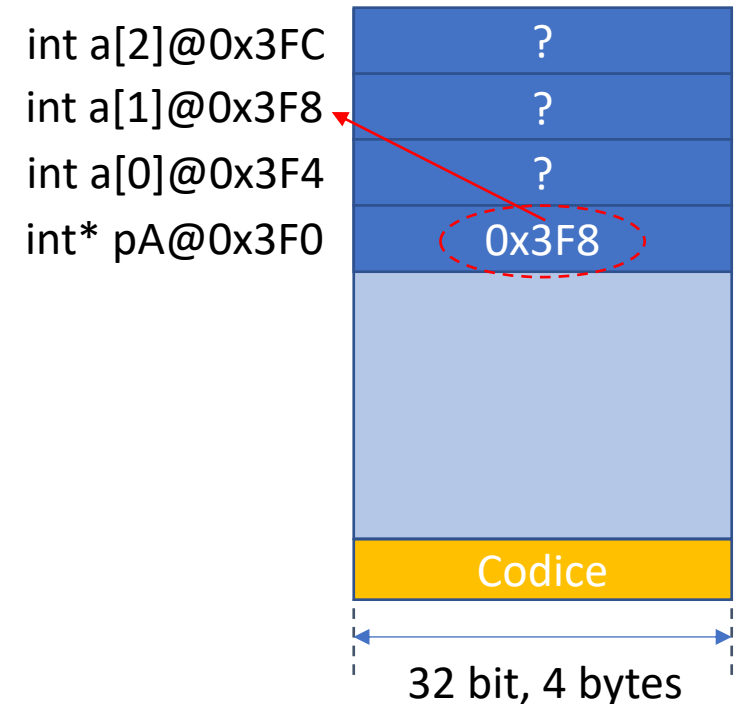
- Gli indirizzi sono tipi numerici positivi interi sui quali sono definiti gli operatori + e -
Ipoteizzando che l'indirizzo di a[0] sia copiato in pA

$\text{int}^* \text{pA} = \text{a};$

l'indirizzo di a[1] in linguaggio C si calcola come

$\text{pA} = \text{pA} + 1;$

- Contrariamente agli altri tipi numerici, per il tipo int^* il risultato dell'operazione per l'esempio sopra $0x3F4 + 1$ vale $0x3F8$



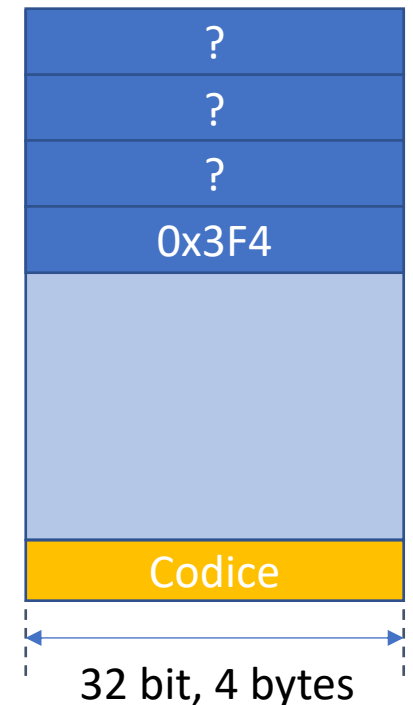
Accesso agli altri elementi dell'array

- Per i tipi puntatore, il linguaggio C definisce le operazioni elementari come multiple della dimensione della parola del tipo puntato
- Nell'esempio, l'addendo 1 a pA è moltiplicato *di nascosto* per la dimensione di un int

$$pA = pA + (1 * \text{sizeof(int)})$$

- Tale comportamento si spiega con la volontà di prevenire accessi non allineati alla memoria

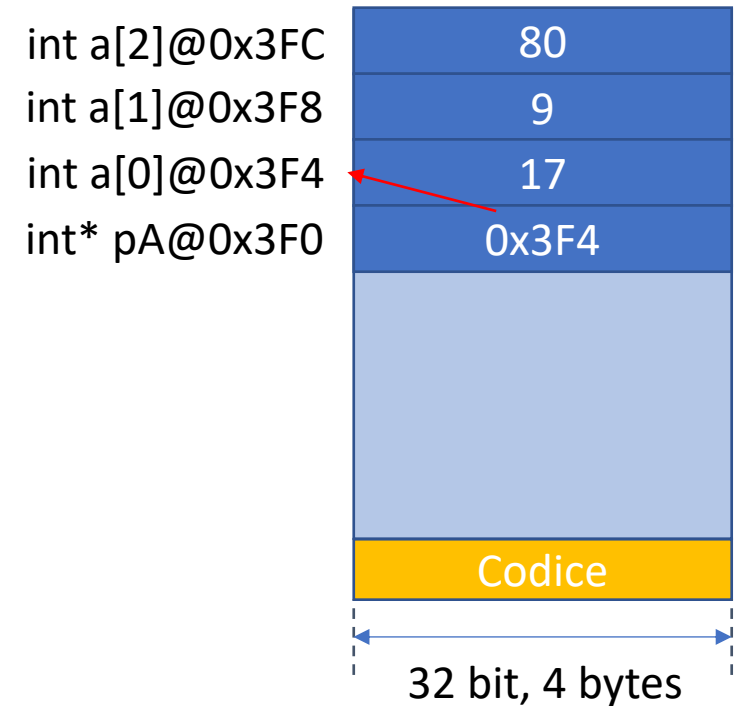
```
int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4
int* pA@0x3F0
```



Aritmetica dei puntatori

- Quindi, data la copia dell'indirizzo di `a[0]` come

`int* pA0 = a = 0x3F4`



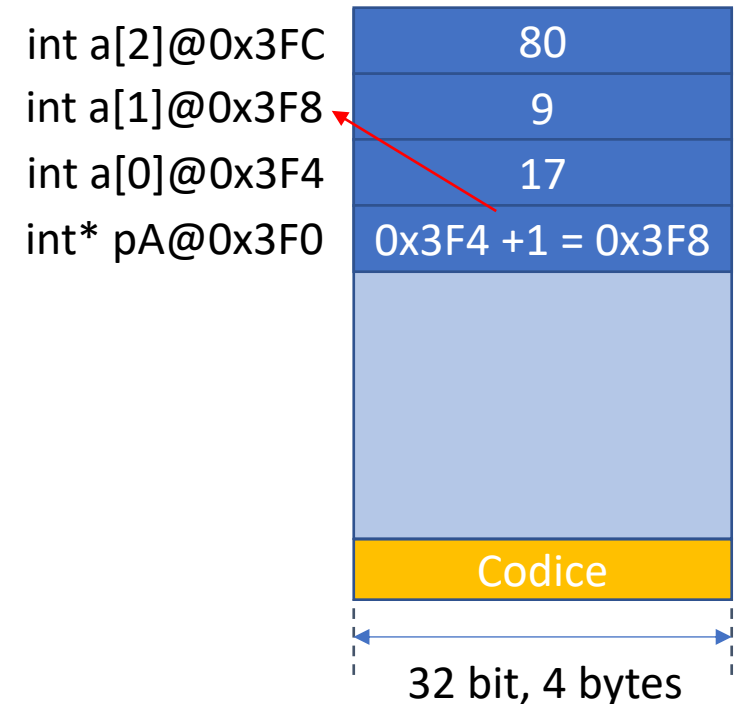
Aritmetica dei puntatori

- Quindi, data la copia dell'indirizzo di `a[0]` come

`int* pA0 = a = 0x3F4`

- L'indirizzo di `a[1]` si calcola come

`pA1 = pA0 + 1 = 0x3F8`



Aritmetica dei puntatori

- Quindi, data la copia dell'indirizzo di `a[0]` come

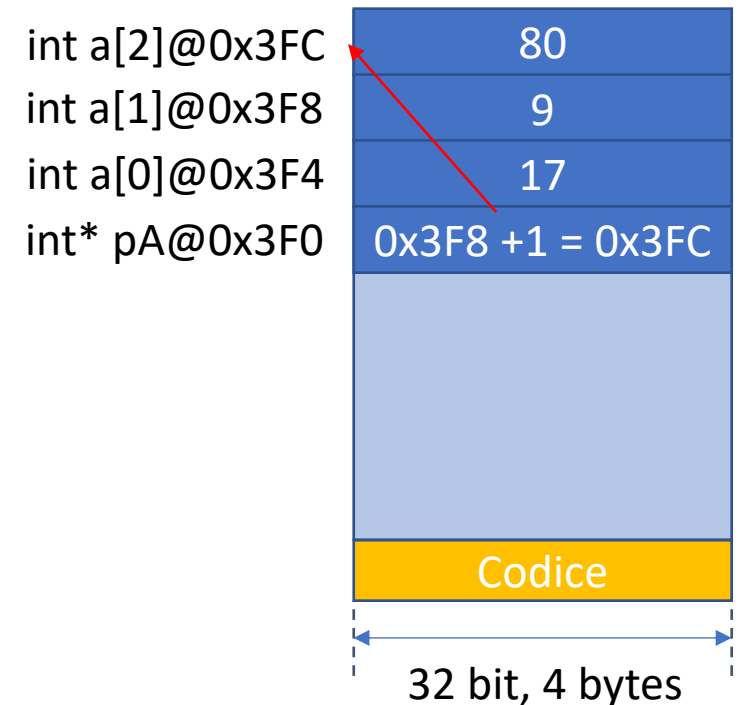
`int* pA0 = a = 0x3F4`

- L'indirizzo di `a[1]` si calcola come

`pA1 = pA0 + 1 = 0x3F8`

- L'indirizzo di `a[2]` si calcola come

`pA2 = pA1 + 1 = 0x3FC`



Aritmetica dei puntatori

- Quindi, data la copia dell'indirizzo di `a[0]` come

`int* pA0 = a = 0x3F4`

- L'indirizzo di `a[1]` si calcola come

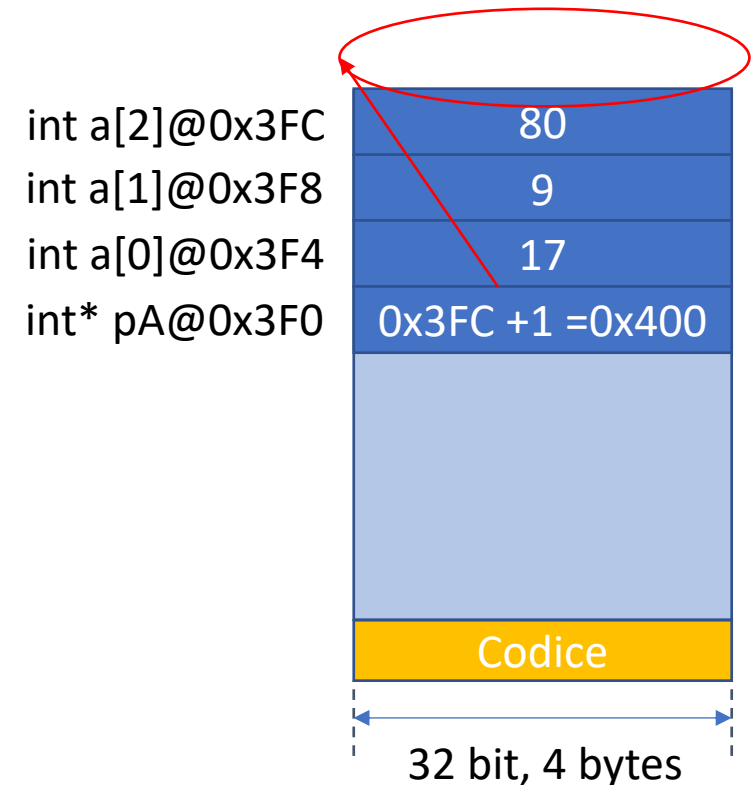
`pA1 = pA0 + 1 = 0x3F8`

- L'indirizzo di `a[2]` si calcola come

`pA2 = pA1 + 1 = 0x3FC`

- Attenzione agli overflows!

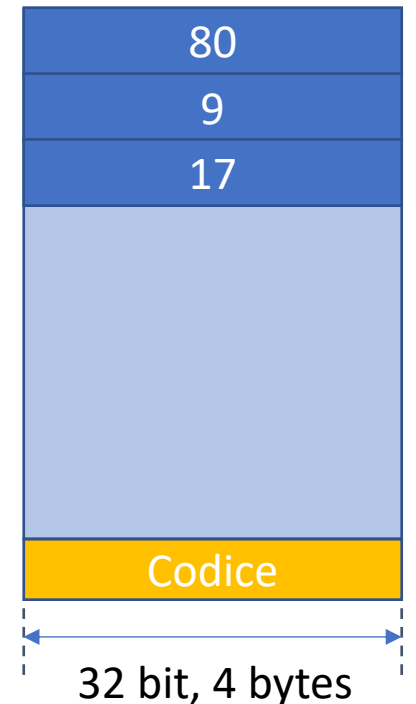
`pA3 = pA2 + 1 = 0x400`



Accesso agli elementi dell'array

```
#define A_LEN 3
int main(void) {
    → int a[A_LEN] = {17, 9, 80};
      int* pA = a;
      int i = 0;
      while (i < A_LEN) {
          printf("a[%d] vale %d\n", i, *pA);
          i++;
          pA++;
      }
}
```

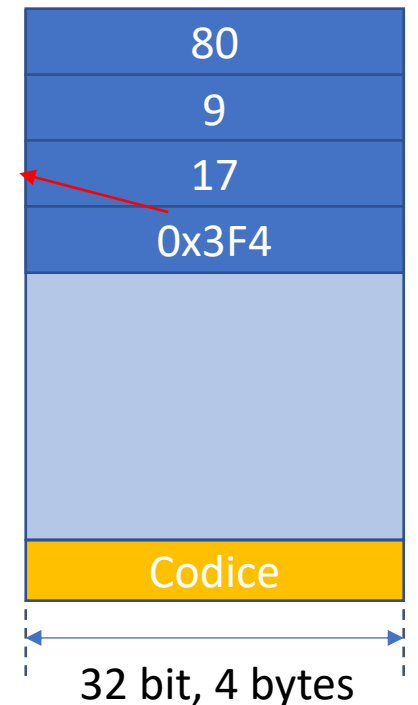
int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4



Accesso agli elementi dell'array

```
#define A_LEN 3
int main(void) {
    int a[A_LEN] = {17, 9, 80};
    int* pA = a;
    int i = 0;
    while (i < A_LEN) {
        printf("a[%d] vale %d\n", i, *pA);
        i++;
        pA++;
    }
}
```

int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4
int* pA@0x3F0

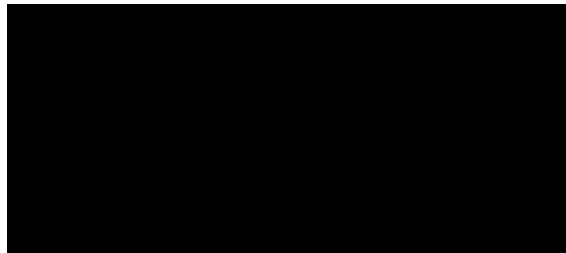
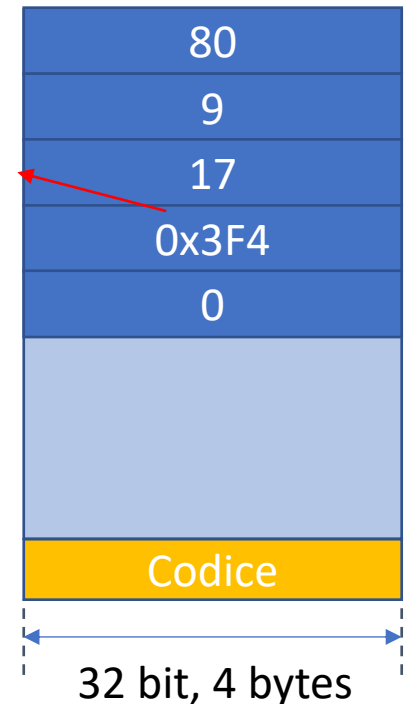


Accesso agli elementi dell'array

```
#define A_LEN 3
int main(void) {
    int a[A_LEN] = {17, 9, 80};
    int* pA = a;
    int i = 0;
    while (i < A_LEN) {
        printf("a[%d] vale %d\n", i, *pA);
        i++;
        pA++;
    }
}
```



int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4
int* pA@0x3F0
int i

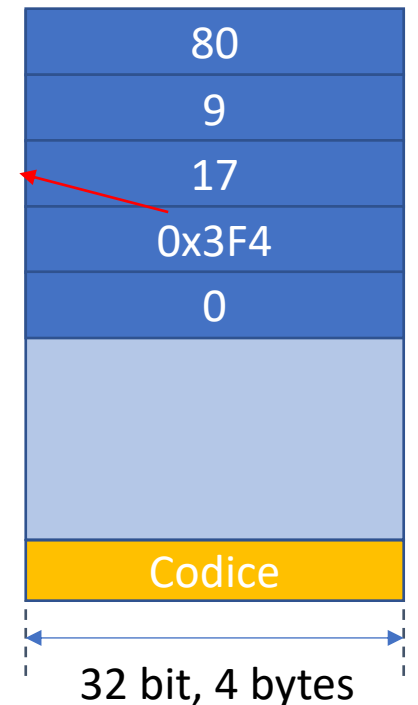


Accesso agli elementi dell'array

```
#define A_LEN 3
int main(void) {
    int a[A_LEN] = {17, 9, 80};
    int* pA = a;
    int i = 0;
    while (i < A_LEN) {
        printf("a[%d] vale %d\n", i, *pA);
        i++;
        pA++;
    }
}
```

➔

int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4
int* pA@0x3F0
int i



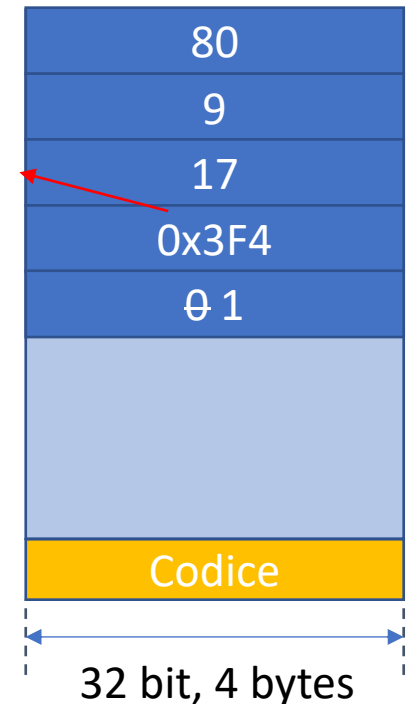
a[0] vale 17

Accesso agli elementi dell'array

```
#define A_LEN 3
int main(void) {
    int a[A_LEN] = {17, 9, 80};
    int* pA = a;
    int i = 0;
    while (i < A_LEN) {
        printf("a[%d] vale %d\n", i, *pA);
        i++;
        pA++;
    }
}
```



int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4
int* pA@0x3F0
int i



a[0] vale 17

Accesso agli elementi dell'array

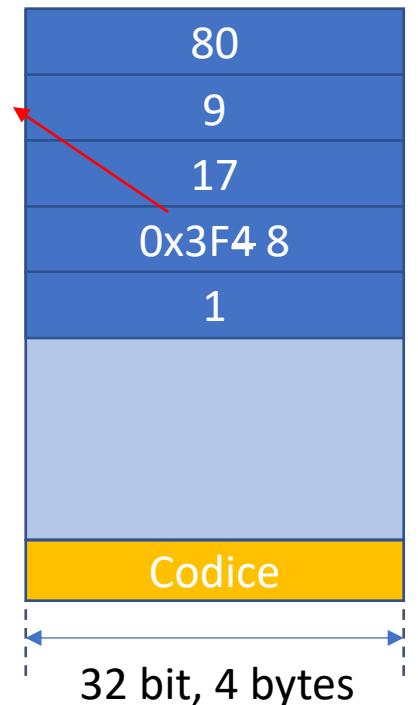
```
#define A_LEN 3
int main(void) {
    int a[A_LEN] = {17, 9, 80};
    int* pA = a;
    int i = 0;
    while (i < A_LEN) {
        printf("a[%d] vale %d\n", i, *pA);
        i++;
        pA++;
    }
}
```



Incrementato
da 0x3F4
a 0x3F8

a[0] vale 17

int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4
int* pA@0x3F0
int i

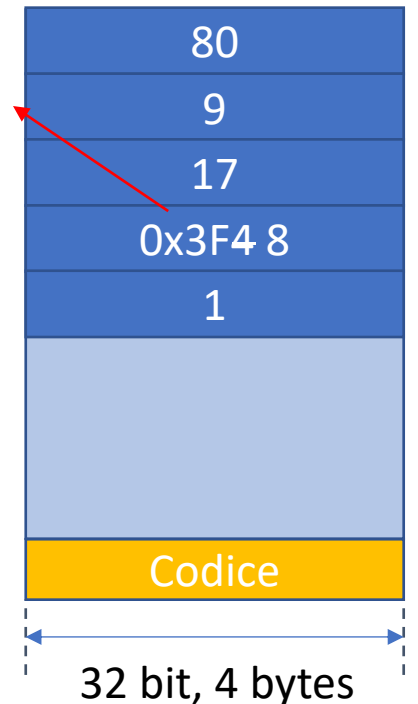


Accesso agli elementi dell'array

```
#define A_LEN 3
int main(void) {
    int a[A_LEN] = {17, 9, 80};
    int* pA = a;
    int i = 0;
    while (i < A_LEN) {
        printf("a[%d] vale %d\n", i, *pA);
        i++;
        pA++;
    }
}
```



int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4
int* pA@0x3F0
int i



a[0] vale 17
a[1] vale 9

Accesso agli elementi dell'array

```
#define A_LEN 3
int main(void) {
    int a[A_LEN] = {17, 9, 80};
    int* pA = a;
    int i = 0;
    while (i < A_LEN) {
        printf("a[%d] vale %d\n", i, *pA);
        i++;
        pA++;
    }
}
```

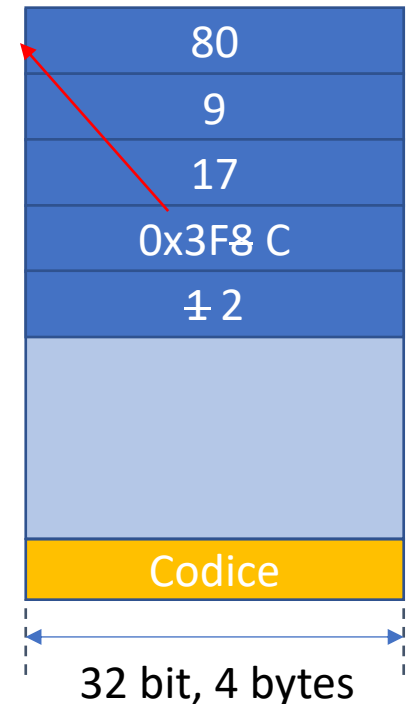


Incrementato
da 0x3F8
a 0x3FC



a[0] vale 17
a[1] vale 9

int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4
int* pA@0x3F0
int i

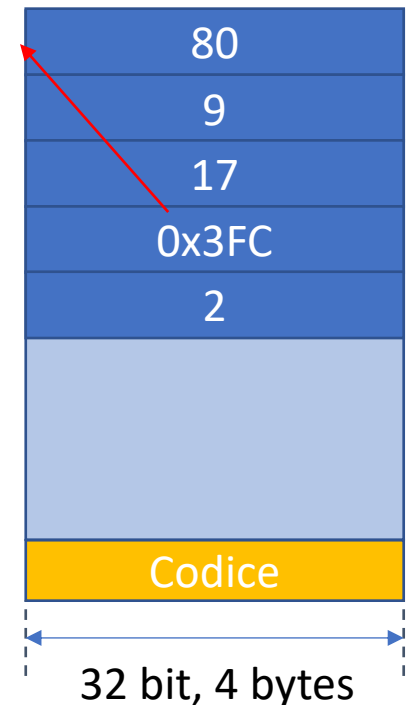


Accesso agli elementi dell'array

```
#define A_LEN 3
int main(void) {
    int a[A_LEN] = {17, 9, 80};
    int* pA = a;
    int i = 0;
    while (i < A_LEN) {
        printf("a[%d] vale %d\n", i, *pA);
        i++;
        pA++;
    }
}
```

➔

int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4
int* pA@0x3F0
int i



a[0] vale 17
a[1] vale 9
a[2] vale 80

Accesso agli elementi dell'array

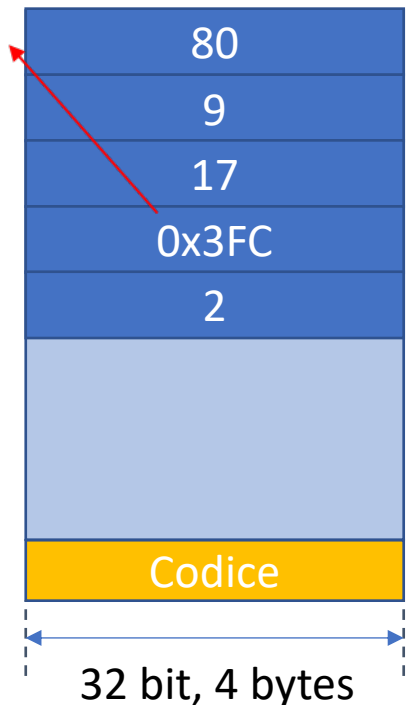
```
#define A_LEN 3
```

```
int main(void) {  
    int a[A_LEN] = {17, 9, 80};  
    int* pA = a;
```

Ciclo *for()*

```
    for (int i = 0; i < A_LEN; i++, pA++) {  
        printf("a[%d] vale %d\n", i, *pA);  
    }  
}
```

int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4
int* pA@0x3F0
int i



a[0] vale 17
a[1] vale 9
a[2] vale 80

Accesso agli elementi dell'array

```
#define A_LEN 3
```

```
int main(void) {
```

```
    int a [A_LEN]={17, 9, 80};
```

```
    for (int i = 0, int* pA = b; i<A_LEN;
```

```
    i++, pA++) {
```

```
        printf("a[%d] vale %d\n", i , *pA);
```

```
    }
```

```
}
```

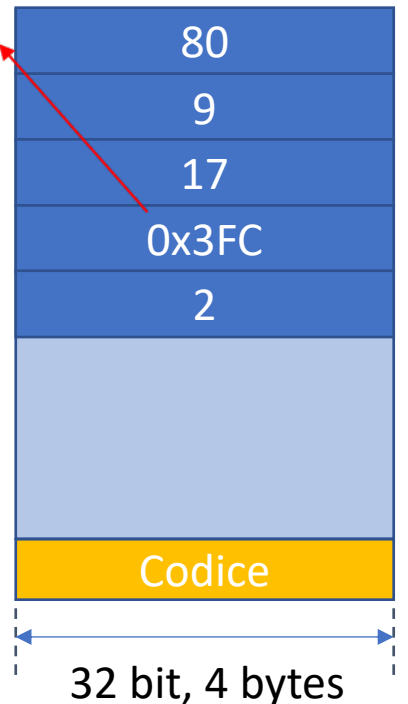
int a[2]@0x3FC

int a[1]@0x3F8

int a[0]@0x3F4

int* pA@0x3F0

int i



a[0] vale 17

a[1] vale 9

a[2] vale 80

Aritmetica dei puntatori

- L'indirizzo dell'elemento i-esimo si calcola come

$$\text{int}^* \text{pA} = \text{a} + \text{i}$$

- Utile per accedere direttamente all'elemento i-esimo dell'array
- Esempi
 - stampa dell'array in ordine inverso
 - ciclo su array aggiornando un puntatore anzichè indice

int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4
int* pA@0x3F0



Accesso agli elementi dell'array

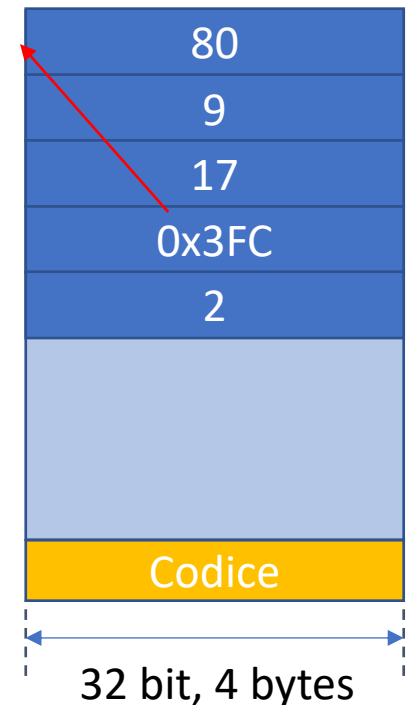
```
# define A_LEN 3
```

```
int main(void) {  
    int a[A_LEN]={17, 9, 80};  
    int* pLast = a + A_LEN - 1;
```

```
    for (int i = A_LEN-1; i>0;i--, pLast--) {  
        printf("a[%d] vale %d\n", i , *pLast);  
    }  
}
```

Indirizzo
dell'ultimo
elemento
dell'array

int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4
int* pLast@0x3F0
int i



a[2] vale 80

Accesso agli elementi dell'array

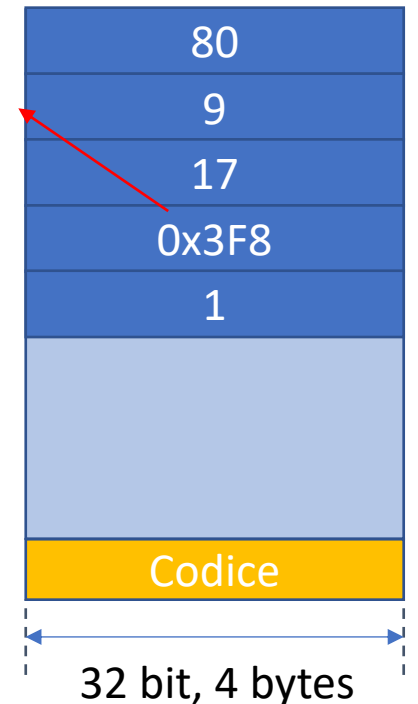
```
# define A_LEN 3
```

```
int main(void) {  
    int a[A_LEN]={17, 9, 80};  
    int* pLast = a + A_LEN - 1;
```

➔

```
    for (int i = A_LEN-1; i>0; i--, pLast--) {  
        printf("a[%d] vale %d\n", i , *pLast);  
    }  
}
```

int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4
int* pLast@0x3F0
int i



a[2] vale 80
a[1] vale 9

Accesso agli elementi dell'array

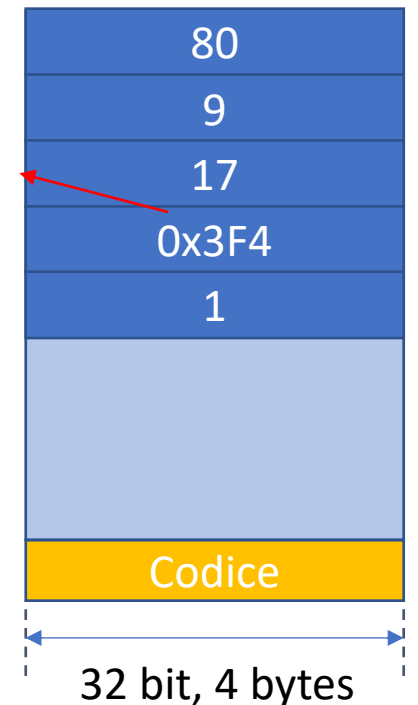
```
# define A_LEN 3
```

```
int main(void) {  
    int a[A_LEN]={17, 9, 80};  
    int* pLast = a + A_LEN - 1;
```

➔

```
    for (int i = A_LEN-1; i>0;i--, pLast--) {  
        printf("a[%d] vale %d\n", i , *pLast);  
    }  
}
```

int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4
int* pLast@0x3F0
int i



a[2] vale 80
a[1] vale 9
a[0] vale 17

Aritmetica dei puntatori

- L'indirizzo dell'elemento i-esimo si calcola come

$$\text{int}^* \text{ pA} = \text{a} + \text{i} ;$$

- Utile per accedere direttamente all'elemento i-esimo dell'array
- Esempi
 - stampa dell'array in ordine inverso
 - ciclo su array aggiornando un puntatore anzichè indice

int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4
int* pA@0x3F0



Accesso agli elementi dell'array

```
#define A_LEN 3
```

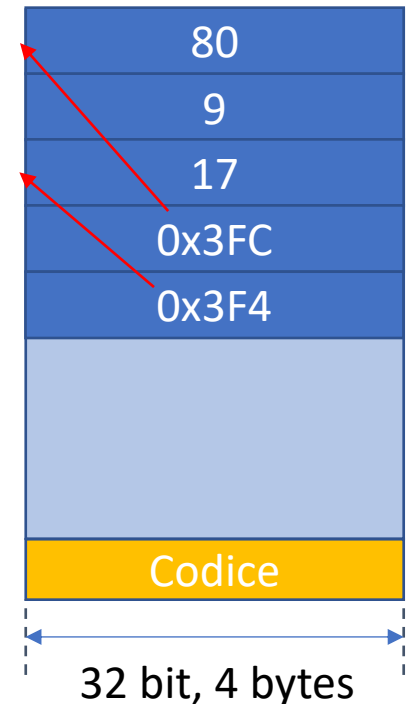
```
int main(void) {  
    int a[A_LEN] = {17, 9, 80};  
    int* pLast = a + A_LEN - 1;
```

Copia di lavoro
indirizzo del
primo elemento
dell'array

```
int a[2]@0x3FC  
int a[1]@0x3F8  
int a[0]@0x3F4  
int* pLast@0x3F0  
int* pA@0x3EC
```

➔

```
    for (int* pA = a; pA <= pLast; pA++) {  
        printf("%d\n", *pA);  
    }  
}
```



Accesso agli elementi dell'array

```
#define A_LEN 3
```

```
int main(void) {
```

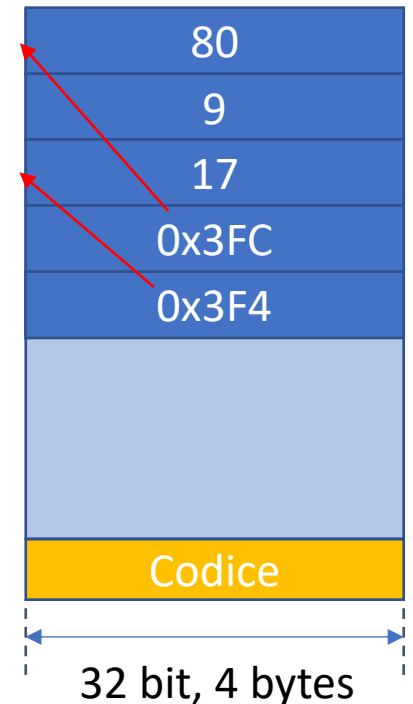
```
    int a[A_LEN] = {17, 9, 80};  
    int* pLast = a + A_LEN - 1;
```

Confronto
fra
puntatori

➔

```
    for (int* pA = a; pA <= pLast; pA++) {  
        printf("%d\n", *pA);  
    }  
}
```

int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4
int* pLast@0x3F0
int* pA@0x3EC



Accesso agli elementi dell'array

```
#define A_LEN 3
```

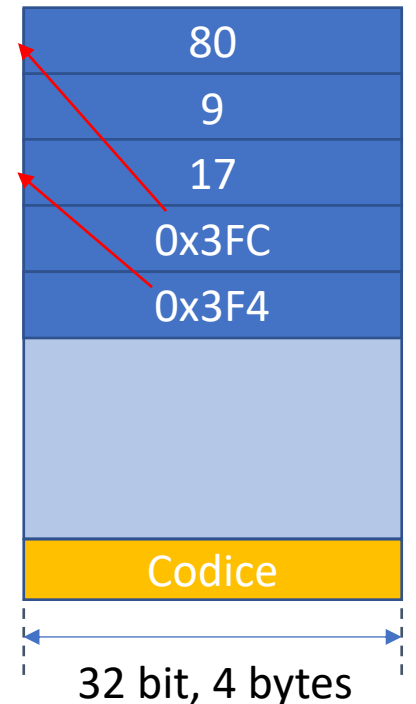
```
int main(void) {  
    int a[A_LEN] = {17, 9, 80};  
    int* pLast = a + A_LEN - 1;
```

```
    for (int* pA = a; pA <= pLast; pA++) {  
        printf("%d\n", *pA);  
    }  
}
```



17

int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4
int* pLast@0x3F0
int* pA@0x3EC



Accesso agli elementi dell'array

```
#define A_LEN 3
```

```
int main(void) {
```

```
    int a[A_LEN] = {17, 9, 80};
```

```
    int* pLast = a + A_LEN - 1;
```

```
    for (int* pA = a; pA <= pLast; pA++ ) {  
        printf("%d\n", *pA);
```

```
    }
```

```
}
```

int a[2]@0x3FC

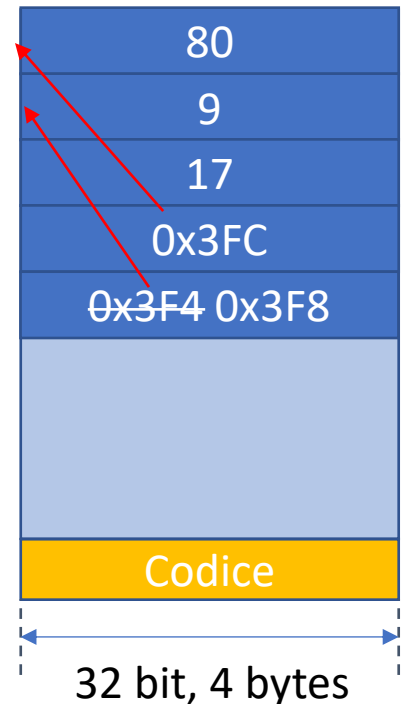
int a[1]@0x3F8

int a[0]@0x3F4

int* pLast@0x3F0

int* pA@0x3EC

Incremento
puntatore pA
di 4 bytes



17

Accesso agli elementi dell'array

```
#define A_LEN 3
```

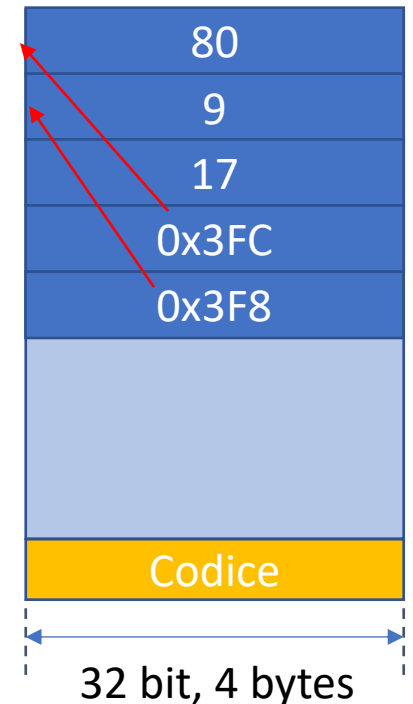
```
int main(void) {  
    int a[A_LEN] = {17, 9, 80};  
    int* pLast = a + A_LEN - 1;
```

Confronto
fra
puntatori

➔

```
    for (int* pA = a; pA <= pLast; pA++) {  
        printf("%d\n", *pA);  
    }  
}
```

```
int a[2]@0x3FC  
int a[1]@0x3F8  
int a[0]@0x3F4  
int* pLast@0x3F0  
int* pA@0x3EC
```



17

Accesso agli elementi dell'array

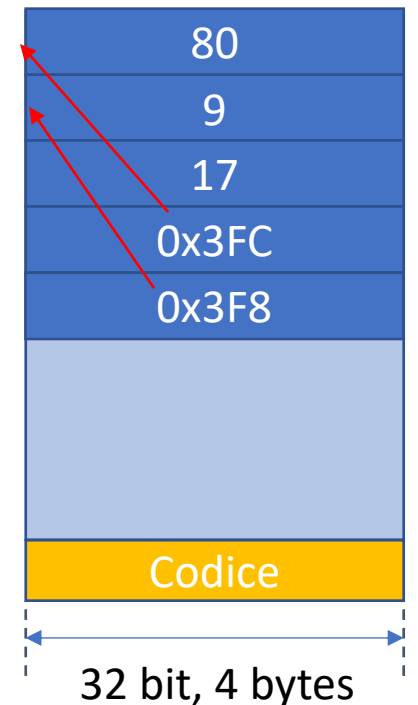
```
#define A_LEN 3
```

```
int main(void) {  
    int a[A_LEN] = {17, 9, 80};  
    int* pLast = a + A_LEN - 1;
```

```
    for (int* pA = a; pA <= pLast; pA++) {  
        printf("%d\n", *pA);  
    }  
}
```



```
int a[2]@0x3FC  
int a[1]@0x3F8  
int a[0]@0x3F4  
int* pLast@0x3F0  
int* pA@0x3EC
```



```
17  
9
```

Accesso agli elementi dell'array

```
#define A_LEN 3
```

```
int main(void) {  
    int a[A_LEN] = {17, 9, 80};  
    int* pLast = a + A_LEN - 1;
```

```
    for (int* pA = a; pA <= pLast; pA++) {  
        printf("%d\n", *pA);
```



```
}
```

```
}
```

```
int a[2]@0x3FC  
int a[1]@0x3F8  
int a[0]@0x3F4  
int* pLast@0x3F0  
int* pA@0x3EC
```



```
17  
9
```

Accesso agli elementi dell'array

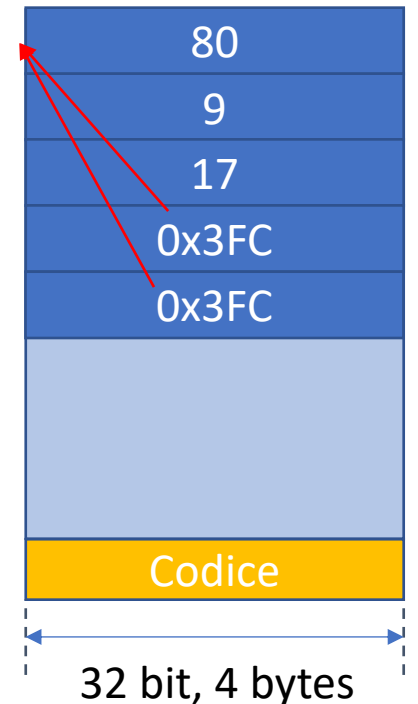
```
#define A_LEN 3
```

```
int main(void) {  
    int a[A_LEN] = {17, 9, 80};  
    int* pLast = a + A_LEN - 1;
```

➔

```
    for (int* pA = a; pA <= pLast; pA++) {  
        printf("%d\n", *pA);  
    }  
}
```

```
int a[2]@0x3FC  
int a[1]@0x3F8  
int a[0]@0x3F4  
int* pLast@0x3F0  
int* pA@0x3EC
```



```
17  
9
```

Accesso agli elementi dell'array

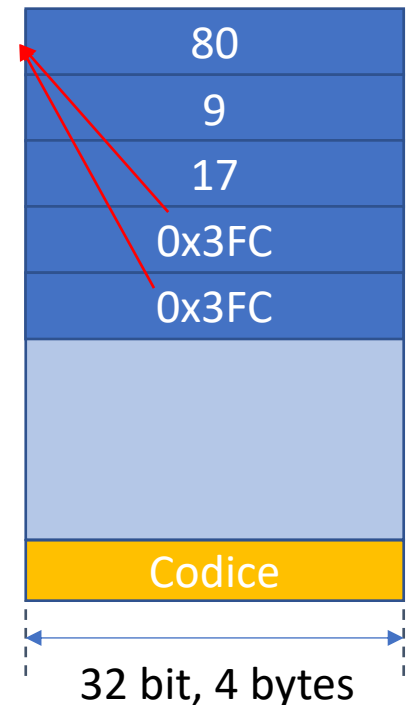
```
#define A_LEN 3
```

```
int main(void) {  
    int a[A_LEN] = {17, 9, 80};  
    int* pLast = a + A_LEN - 1;
```

```
    for (int* pA = a; pA <= pLast; pA++) {  
        printf("%d\n", *pA);  
    }  
}
```



```
int a[2]@0x3FC  
int a[1]@0x3F8  
int a[0]@0x3F4  
int* pLast@0x3F0  
int* pA@0x3EC
```



```
17  
9  
80
```

Accesso agli elementi dell'array

```
#define A_LEN 3
```

```
int main(void) {  
    int a[A_LEN] = {17, 9, 80};  
    int* pLast = a + A_LEN - 1;
```

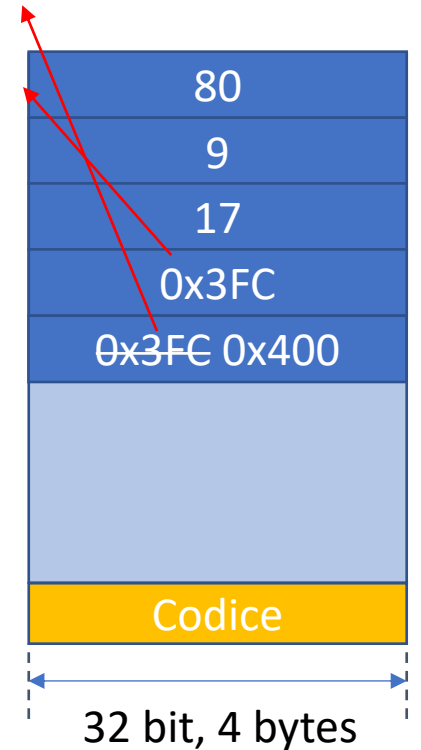
```
    for (int* pA = a; pA <= pLast; pA++) {  
        printf("%d\n", *pA);
```



```
    }
```

```
}
```

```
int a[2]@0x3FC  
int a[1]@0x3F8  
int a[0]@0x3F4  
int* pLast@0x3F0  
int* pA@0x3EC
```



```
17  
9  
80
```

Accesso agli elementi dell'array

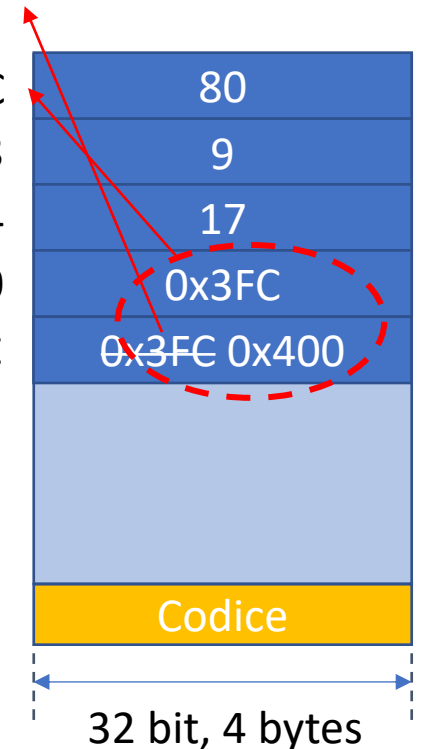
```
#define A_LEN 3
```

```
int main(void) {  
    int a[A_LEN] = {17, 9, 80};  
    int* pLast = a + A_LEN - 1;
```

➔

```
    for (int* pA = a; pA <= pLast; pA++) {  
        printf("%d\n", *pA);  
    }  
}
```

```
int a[2]@0x3FC  
int a[1]@0x3F8  
int a[0]@0x3F4  
int* pLast@0x3F0  
int* pA@0x3EC
```



```
17  
9  
80
```

Accesso agli elementi dell'array

```
#define A_LEN 3
```

```
int main(void) {  
    int a[A_LEN] = {17, 9, 80};  
    int* pLast = a + A_LEN - 1;
```

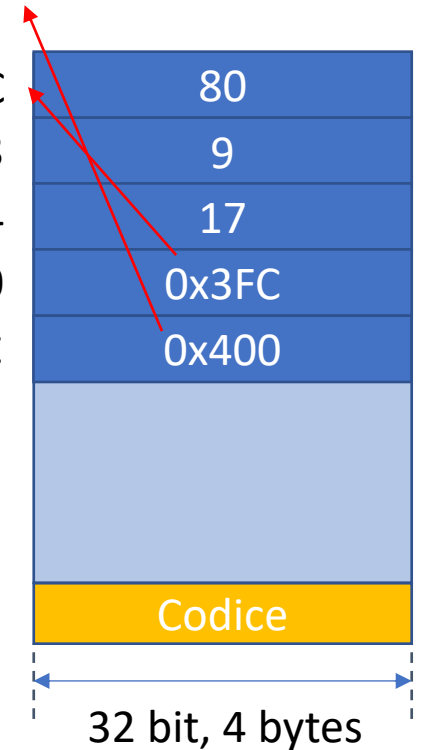
```
    for (int* pA = a; pA <= pLast; pA++) {  
        printf("%d\n", *pA);
```



```
    }
```

```
}
```

```
int a[2]@0x3FC  
int a[1]@0x3F8  
int a[0]@0x3F4  
int* pLast@0x3F0  
int* pA@0x3EC
```



```
17  
9  
80
```

Passaggio di array come parametri

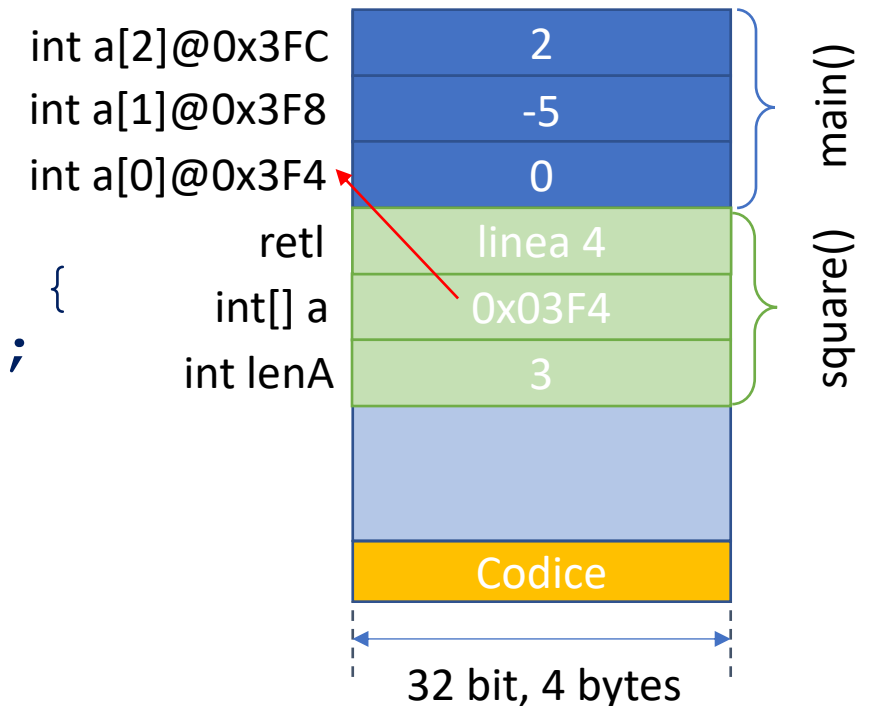
- Possediamo tutti gli elementi per riscrivere la funzione per il calcolo del quadrato degli elementi di un array senza parentesi «[]»
 - la funzione sarà dichiarata ricevere un puntatore `int*` anzichè `int[]`
 - calcoleremo a mano gli indirizzi di ogni elemento dell array
 - accederemo al valore di tale elemento via dereferenziazione «*»
- È un esercizio dal valore didattico, nella realtà useremo le «[]» nel resto di questo insegnamento
- Tuttavia è frequente indirizzare aree di memoria come vedremo in seguito

Passaggio di array come parametri con []

```
#define SIZE_A 3
```

```
void main(void) {  
    int a[] = {0, -5, 2};  
    square (a, SIZE_A);  
    for (int i = 0; i < SIZE_A; i++) {  
        printf("%d\t%d\n", i, a[i]);  
    }  
}
```

```
void square (int a[], int lenA) {  
    for (int i = 0; i < lenA; i++) {  
        a[i] = a[i] * a[i];  
    }  
}
```



Passaggio di array come parametri con *tipo**

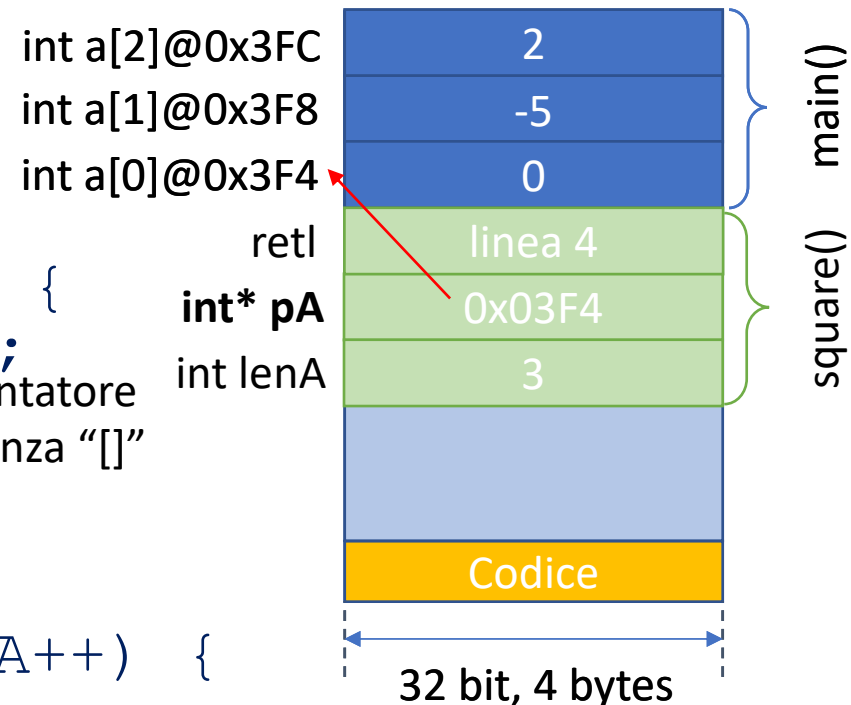
```
#define SIZE_A 3
```

```
void main(void) {  
    int a[] = {0, -5, 2};  
    square(a, SIZE_A);  
    for (int i = 0; i < SIZE_A; i++) {  
        printf("%d\t%d\n", i, a[i]);  
    }  
}
```

➔

```
void square(int* pA, int lenA) {  
    for (int i = 0; i < lenA; i++, pA++) {  
        *pA = *pA * *pA;  
    }  
}
```

pA è un puntatore
base di a senza "[]"

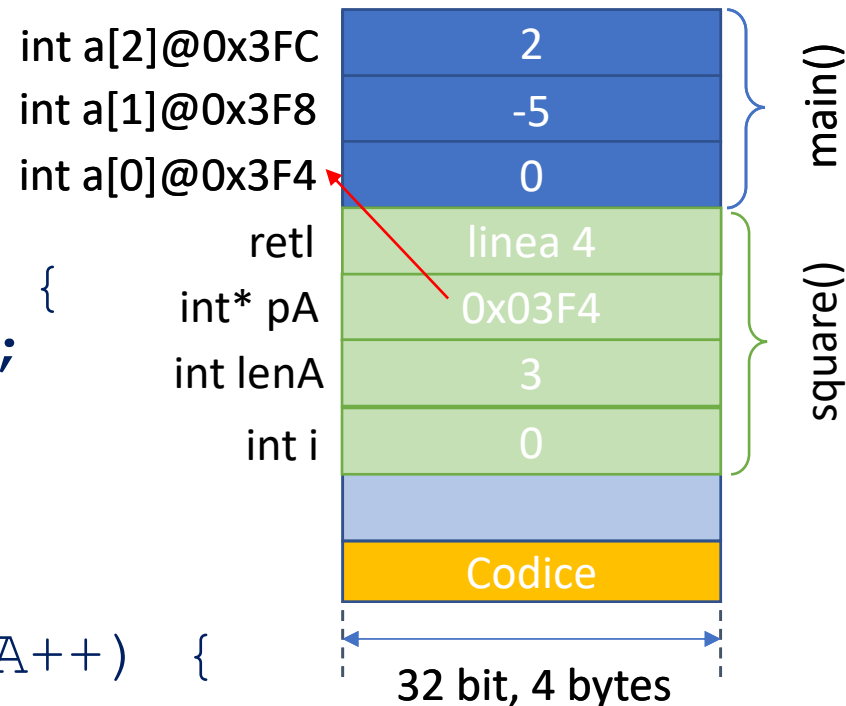


Passaggio di array come parametri con *tipo**

```
#define SIZE_A 3
```

```
void main(void) {  
    int a[] = {0, -5, 2};  
    square(a, SIZE_A);  
    for (int i = 0; i < SIZE_A; i++) {  
        printf("%d\t%d\n", i, a[i]);  
    }  
}
```

```
void square(int* pA, int lenA) {  
    for (int i = 0; i < lenA; i++, pA++) {  
        *pA = *pA * *pA;  
    }  
}
```



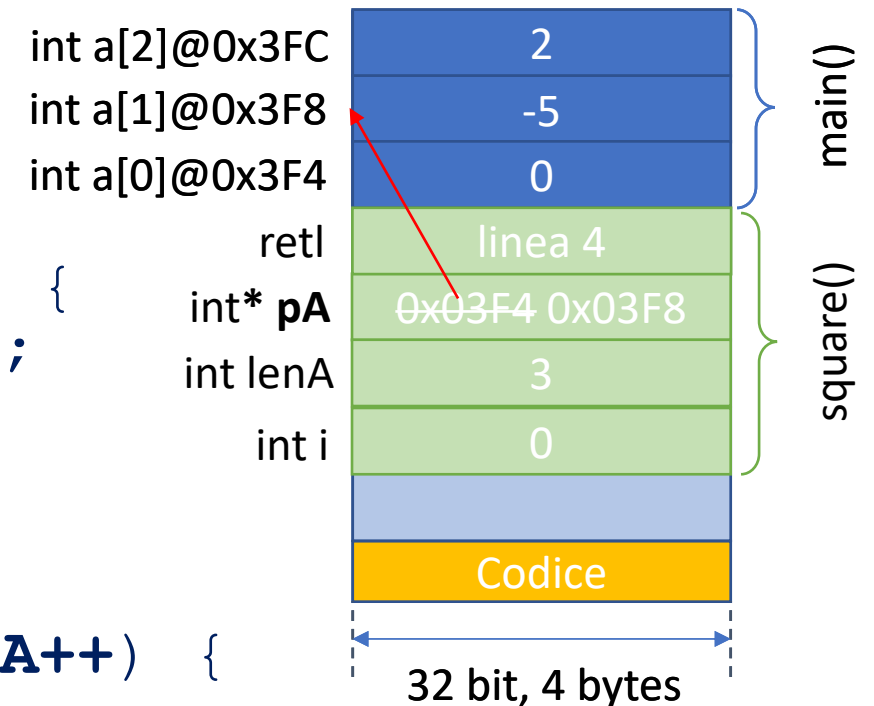
tre operatori
dereferenziazione “*”

Passaggio di array come parametri con *tipo**

```
#define SIZE_A 3
```

```
void main(void) {  
    int a[] = {0, -5, 2};  
    square(a, SIZE_A);  
    for (int i = 0; i < SIZE_A; i++) {  
        printf("%d\t%d\n", i, a[i]);  
    }  
}
```

```
void square(int* pA, int lenA) {  
    for (int i = 0; i < lenA; i++, pA++) {  
        *pA = *pA * *pA;  
    }  
}
```



Incrementa
pA di 4 bytes



Indicizzazione aree di memoria

- Le parentesi «[]» possono essere sfruttate per accedere a una generica area di memoria che rispetti che i 4 requisiti sopra
- Finora abbiamo allocato aree di memoria esplicitamente per contenere array di dati primitivi
- Nella realtà della programmazione in C, è molto comune allocare un'area di memoria senza nozione preliminare di array
- Noi simuleremo questa situazione allocando un'area di memoria di 12 bytes come 3 variabili di tipo int contigue in memoria
- È un esempio didattico, **da non prendere a modello**

Array sbagliato e operatore []

DANGER

NON USARE!

```
#include <stdio.h>
#define A_LEN 3
```

```
int main(void){
    int* p = NULL;
```

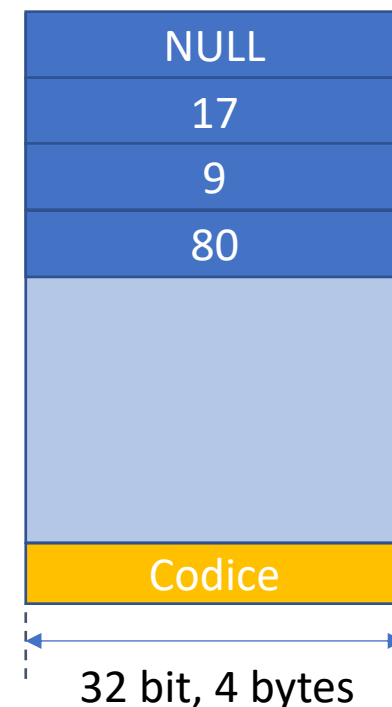
➔

```
{
    int x = 17;
    int y = 9;
    int z = 80;
```

```
    p = &z;
    for(int i = 0; i < A_LEN; i++) {
        printf("p[%d] vale %d\n", i , p[i]);
    }
}
```

12 bytes
contigui
in memoria

int* p@0x3FC
int x@0x3F8
int y@0x3F4
int z@0x3F0



Array sbagliato e operatore []

DANGER

NON USARE!

```
#include <stdio.h>
#define A_LEN 3
```

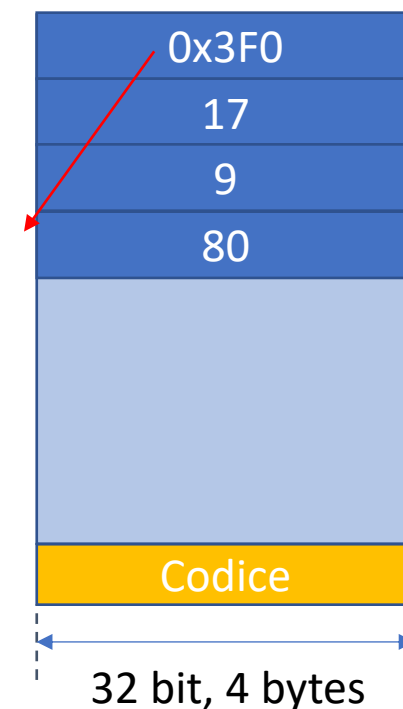
```
int main(void){
    int* p = NULL;
```

```
    int x = 17;
    int y = 9;
    int z = 80;
```



```
    p = &z;
    for(int i = 0; i < A_LEN; i++) {
        printf("p[%d] vale %d\n", i , p[i]);
    }
}
```

```
int* p@0x3FC
int x@0x3F8
int y@0x3F4
int z@0x3F0
```



Array sbagliato e operatore []

DANGER

NON USARE!

```
#include <stdio.h>
#define A_LEN 3
```

```
int main(void){
    int* p = NULL;
```

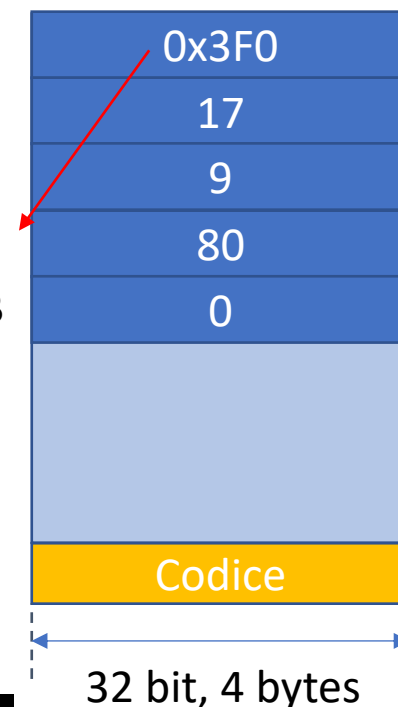
```
    int x = 17;
    int y = 9;
    int z = 80;
```

```
    p = &z;
    for(int i = 0; i < A_LEN; i++) {
        printf("p[%d] vale %d\n", i , p[i]);
    }
```

```
}
```

$$0x3F0 = 0x3F0 + (0 * 4)$$

```
int* p@0x3FC
int x@0x3F8
int y@0x3F4
int z@0x3F0
int i@0x3EB
```



```
$ ./array_sbagliato
P[0] vale 80
```


DANGER

NON USARE!

$$0x3F4 = 0x3F0 + (1 * 4)$$

Diagram illustrating the instruction format (32 bit, 4 bytes):

- 0x3F0
- 17
- 9
- 80
- 1
- Codice

32 bit, 4 bytes

79

Array sbagliato e operatore []

DANGER

NON USARE!

```
#include <stdio.h>
#define A_LEN 3
```

```
int main(void){
    int* p = NULL;
```

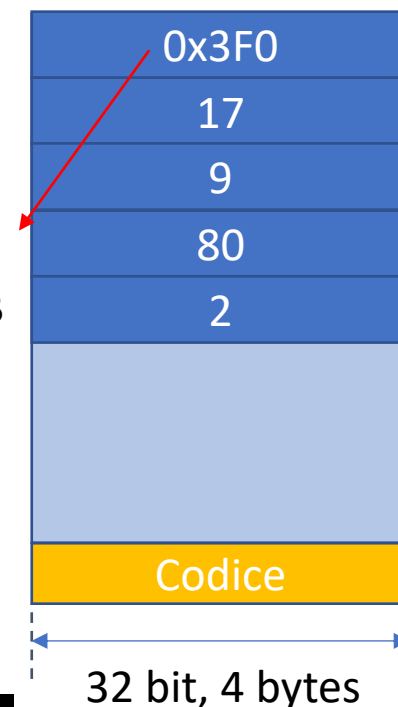
```
    int x = 17;
    int y = 9;
    int z = 80;
```

```
    p = &z;
    for(int i = 0; i < A_LEN; i++) {
        printf("p[%d] vale %d\n", i , p[i]);
    }
```

```
}
```

$$0x3F8 = 0x3F0 + (2 * 4)$$

```
int* p@0x3FC
int x@0x3F8
int y@0x3F4
int z@0x3F0
int i@0x3EB
```



```
$ ./array_sbagliato
P[0] vale 80
P[1] vale 9
P[2] vale 17
```

Indicizzazione aree di memoria

- Abbiamo visto 4 casi per gestire aree di memoria composte da elementi di tipo omogeneo allocati ad indirizzi contigui
 1. Area dichiarata come array, accesso tramite «[]»
 2. Area dichiarata come array, accesso tramite «int*» e «*»
 3. Area generica di memoria, accesso tramite «int*» e «*»
 4. Area generica di memoria, accesso tramite «[]»
- Le opzioni sopra sono tutte possibili, la scelta dipenderà dai vincoli di progetto